



Yenepoya (Deemed To Be University)

PROJECT –LOW LEVEL DESIGN
ON

Predictive Modeling for Retail Success

Student Name: Saheen

23BCDSBA17

Industry Mentor:
Mr.Ujjwal Kumar

Guided by:
Ms.Raksha Adyanthaya

Table of Contents

1. Introduction	1
2. 1.1 Purpose of the Document	1
3. 1.2 Scope	1
4. System Architecture	2
5. Module Design	2
6. Data Structures	3
7. Algorithms	3
8. API Design	4
9. Error Handling	4
10. Logging	5
11. Security	5
12. Performance	6
13. Deployment	6
14. Future Enhancements	7

1. Introduction

This document presents the Low Level Design (LLD) of the Customer Churn Prediction System for Predictive Modeling for Retail Success. While the High Level Design described the overall architecture and system components, this LLD elaborates on the internal design of each component, including class structures, function signatures, data flow logic, algorithm pseudocode, and database schemas.

The purpose of this document is to provide developers and data scientists with the precise technical specifications required to implement the system. This document serves as a direct bridge between the HLD and the actual code implementation.

1.1. Scope of the Document

This Low Level Design document provides detailed specifications for every module of the Customer Churn Prediction System. It covers:

- Detailed class and function definitions for all system modules
- Algorithm pseudocode for preprocessing, feature engineering, and model training
- Database schema and data field specifications
- Internal component interactions and method-level information flow
- Error handling strategies and edge case management
- Unit test cases per module

This document is restricted to implementation-level details and does not re-describe high-level architectural decisions.

1.2. Intended Audience

This document is intended for software developers, data scientists, and QA engineers who are responsible for implementing, testing, and maintaining the Customer Churn Prediction System. It assumes familiarity with Python, machine learning libraries (scikit-learn, pandas, NumPy), and standard software development practices.

1.3. System Overview

The Customer Churn Prediction System is built in Python using a modular structure. It processes structured tabular customer data through a pipeline of: data ingestion → preprocessing → feature engineering → model training → evaluation → prediction. Each stage is represented by a dedicated Python class or module with well-defined interfaces and responsibilities.

2. Module Designs

2.1. Data Ingestion Module

2.1.1. Class: DataLoader

The DataLoader class is responsible for reading raw data from external sources and returning a validated DataFrame for downstream processing.

Attribute / Method	Type	Description
file_path	str	Path to the CSV dataset file
__init__(self, file_path)	Constructor	Initializes DataLoader with the dataset path
load_data()	DataFrame	Reads CSV file using pandas; raises FileNotFoundError if missing
validate_schema(df)	bool	Checks that required columns exist in the loaded DataFrame
get_shape(df)	tuple	Returns (rows, columns) of the dataset for logging

2.1.2. Pseudocode – load_data()

BEGIN load_data()

- Open file at self.file_path using pandas.read_csv()
- If IOError: raise FileNotFoundError with descriptive message
- Call validate_schema(df) — if False, raise ValueError('Missing required columns')
- Log shape via get_shape(df)
- Return df

END load_data()

2.2. Data Preprocessing Module

2.2.1. Class: DataPreprocessor

The DataPreprocessor class handles all data cleaning operations, ensuring the dataset is free of inconsistencies before entering the feature engineering stage.

Method	Parameters	Return Type	Description
drop_irrelevant_cols()	df: DataFrame	DataFrame	Drops columns: RowNumber, CustomerId, Surname
handle_missing_values()	df: DataFrame	DataFrame	Fills or drops rows with null values per column policy

Method	Parameters	Return Type	Description
remove_duplicates()	df: DataFrame	DataFrame	Removes exact duplicate rows
encode_categoricals()	df: DataFrame	DataFrame	Label-encodes Gender; one-hot encodes Geography
preprocess()	df: DataFrame	DataFrame	Orchestrates all steps above in sequence

2.2.2. Pseudocode – preprocess()

BEGIN preprocess(df)

- $df \leftarrow \text{drop_irrelevant_cols}(df)$
- $df \leftarrow \text{handle_missing_values}(df)$ // strategy: median for numerics, mode for categorical
- $df \leftarrow \text{remove_duplicates}(df)$
- $df \leftarrow \text{encode_categoricals}(df)$ // LabelEncoder for Gender; pd.get_dummies for Geography
- Return df

END preprocess(df)

2.3. Feature Engineering Module

2.3.1. Class: FeatureEngineer

The FeatureEngineer class derives new meaningful features from existing columns to improve model discrimination ability.

Method	Description	Formula / Logic
add_balance_salary_ratio()	Ratio of Balance to EstimatedSalary	$\text{Balance} / (\text{EstimatedSalary} + 1)$
add_age_tenure_ratio()	Ratio of Age to Tenure for behavioral signal	$\text{Age} / (\text{Tenure} + 1)$
add_products_per_tenure()	Product adoption rate over time	$\text{NumOfProducts} / (\text{Tenure} + 1)$
add_is_active_balance()	Interaction between activity and balance	$\text{IsActiveMember} * \text{Balance}$
engineer_features()	Orchestrates all feature additions	Calls all above methods in order

2.4. Model Module

2.4.1. Class: ChurnModel

The ChurnModel class encapsulates model selection, training, hyperparameter management, and serialization.

Attribute / Method	Type / Params	Description
model_type	str	'logistic' or 'decision_tree'
model	sklearn estimator	Holds the instantiated sklearn model object
__init__(self, model_type)	Constructor	Initializes the correct model based on model_type
train(X_train, y_train)	None	Fits model to training data; logs training time
predict(X_test)	ndarray	Returns class labels (0 or 1) for X_test
predict_proba(X_test)	ndarray	Returns probability scores for churn class
save_model(path)	None	Serializes trained model using joblib.dump()
load_model(path)	None	Loads serialized model using joblib.load()

2.4.2. Pseudocode – train()

BEGIN train(X_train, y_train)

- If model_type == 'logistic': self.model = LogisticRegression(max_iter=200, random_state=42)
- Elif model_type == 'decision_tree': self.model = DecisionTreeClassifier(max_depth=5, random_state=42)
- Record start_time
- self.model.fit(X_train, y_train)
- Log elapsed time = current_time - start_time

END train()

2.5. Evaluation Module

2.5.1. Class: ModelEvaluator

Method	Parameters	Output
compute_accuracy()	y_true, y_pred	float – overall accuracy score

Method	Parameters	Output
<code>compute_confusion_matrix()</code>	<code>y_true, y_pred</code>	2x2 ndarray – TP, TN, FP, FN
<code>compute_classification_report()</code>	<code>y_true, y_pred</code>	str – precision, recall, F1 per class
<code>compute_roc_auc()</code>	<code>y_true, y_proba</code>	float – ROC-AUC score
<code>plot_confusion_matrix()</code>	<code>y_true, y_pred</code>	Displays heatmap via matplotlib/seaborn
<code>plot_roc_curve()</code>	<code>y_true, y_proba</code>	Displays ROC curve plot

2.6. Prediction Module

2.6.1. Class: PredictionEngine

The PredictionEngine wraps a trained ChurnModel and applies all preprocessing and feature engineering steps to new single-record inputs before generating predictions.

Method	Parameters	Description
<code>__init__(model_path)</code>	str	Loads serialized ChurnModel from disk
<code>prepare_input(raw_dict)</code>	dict	Converts raw key-value customer record to feature array
<code>predict_churn(raw_dict)</code>	dict → int	Returns 0 (retained) or 1 (churned)
<code>predict_with_confidence(raw_dict)</code>	dict → tuple	Returns (label, probability) for the churn class

2.6.2. Pseudocode – predict_churn()

BEGIN predict_churn(raw_dict)

- `features ← prepare_input(raw_dict)` // apply preprocessing + feature engineering
- `label ← self.model.predict([features])[0]`
- Return label // 0 = not churn, 1 = churn

END predict_churn()

3. Detailed Data Design

3.1. Dataset Schema

The following table defines every field in the Churn Modelling dataset used as the primary data source.

Column Name	Data Type	Nullable	Description
RowNumber	int	No	Sequential row index – dropped during preprocessing
CustomerId	int	No	Unique customer identifier – dropped during preprocessing
Surname	str	No	Customer surname – dropped during preprocessing
CreditScore	int	No	Customer credit score (350 – 850)
Geography	str (cat)	No	Country: France, Germany, Spain
Gender	str (cat)	No	Male / Female
Age	int	No	Customer age in years
Tenure	int	No	Number of years as a bank customer (0–10)
Balance	float	No	Account balance in local currency
NumOfProducts	int	No	Number of bank products held (1–4)
HasCrCard	int (bool)	No	1 if customer has a credit card, else 0
IsActiveMember	int (bool)	No	1 if customer is active, else 0
EstimatedSalary	float	No	Estimated annual salary
Exited	int (bool)	No	Target: 1 = churned, 0 = retained

3.2. Engineered Feature Definitions

Feature Name	Formula	Rationale
balance_salary_ratio	$\text{Balance} / (\text{EstimatedSalary} + 1)$	Captures relative financial exposure
age_tenure_ratio	$\text{Age} / (\text{Tenure} + 1)$	Older customers with short tenure may be at risk
products_per_tenure	$\text{NumOfProducts} / (\text{Tenure} + 1)$	Measures product adoption velocity

Feature Name	Formula	Rationale
is_active_balance	IsActiveMember * Balance	Active customers with high balance are low churn risk

3.3. Train-Test Split Specification

Parameter	Value	Justification
Test size	20% (0.2)	Standard split ratio for medium datasets
Random state	42	Ensures reproducibility across runs
Stratification	By Exited column	Preserves churn class distribution in both splits
Feature matrix (X)	All columns except Exited	Input features for model training
Target vector (y)	Exited column only	Binary churn label for supervised learning

4. Algorithm Specifications

4.1. Logistic Regression

Parameter	Value	Description
solver	lbfgs	Limited-memory BFGS optimizer; suitable for small datasets
max_iter	200	Maximum iterations for convergence
C (regularization)	1.0 (default)	Inverse of regularization strength
random_state	42	Seed for reproducibility
multi_class	auto	Handles binary classification automatically

4.2. Decision Tree Classifier

Parameter	Value	Description
criterion	gini	Gini impurity for split quality measurement
max_depth	5	Limits tree depth to prevent overfitting
min_samples_split	2 (default)	Minimum samples required to split a node
min_samples_leaf	1 (default)	Minimum samples required at a leaf node
random_state	42	Seed for reproducibility

4.3. Scaling Strategy

Numerical features (CreditScore, Age, Balance, EstimatedSalary, and engineered features) are scaled using StandardScaler before training the Logistic Regression model. Decision Tree Classifier does not require scaling as it is invariant to feature magnitudes.

Step	Action
1	Instantiate StandardScaler on training data only (fit on X_train)
2	Transform X_train using fitted scaler
3	Transform X_test using the same fitted scaler (no re-fitting)
4	Store scaler alongside trained model for inference use

5. Error Handling and Validation

5.1. Error Handling Strategy

Each module implements try-except blocks to capture and log exceptions without crashing the pipeline. A centralized logger (Python's logging module at INFO and ERROR levels) records all exceptions with timestamps and module context.

Module	Error Type	Handling Action
DataLoader	FileNotFoundError	Log error; terminate with descriptive message
DataLoader	KeyError (missing columns)	Raise ValueError listing missing fields
DataPreprocessor	ValueError (encoding)	Log warning; skip column and continue
FeatureEngineer	ZeroDivisionError	Add +1 smoothing in all ratio denominators
ChurnModel	ConvergenceWarning	Log warning; increase max_iter and retry once
ModelEvaluator	ValueError (empty predictions)	Raise error if y_pred is empty or all same class
PredictionEngine	FileNotFoundError (model)	Raise IOError if model file does not exist

5.2. Input Validation Rules

Field	Validation Rule	Action on Failure
CreditScore	Integer in range [300, 900]	Raise ValueError with field name
Age	Integer in range [18, 100]	Raise ValueError with field name
Balance	Float ≥ 0	Raise ValueError with field name
NumOfProducts	Integer in [1, 4]	Raise ValueError with field name
Geography	One of: France, Germany, Spain	Raise ValueError with allowed values
Gender	One of: Male, Female	Raise ValueError with allowed values
Tenure	Integer in [0, 10]	Raise ValueError with field name

6. Unit Test Plan

6.1. Test Cases per Module

Test ID	Module	Test Description	Expected Result
TC-01	DataLoader	Load valid CSV file	Returns DataFrame with 14 columns
TC-02	DataLoader	Load non-existent file	Raises FileNotFoundError
TC-03	DataLoader	Load CSV missing 'Exited' column	Raises ValueError
TC-04	DataPreprocessor	Drop RowNumber, CustomerId, Surname	DataFrame has 11 columns
TC-05	DataPreprocessor	Handle row with null Balance	Row is filled with median value
TC-06	DataPreprocessor	Encode Gender column	Values are 0 or 1 only
TC-07	FeatureEngineer	Compute balance_salary_ratio	New column exists; no division by zero
TC-08	ChurnModel	Train logistic model on sample data	Model accuracy > 0.5
TC-09	ChurnModel	Save and reload model	Reloaded model predicts same labels
TC-10	ModelEvaluator	Compute accuracy on known output	Matches manually computed value
TC-11	ModelEvaluator	Confusion matrix shape	Returns 2x2 array
TC-12	PredictionEngine	Predict on valid single record	Returns 0 or 1
TC-13	PredictionEngine	Predict with invalid CreditScore	Raises ValueError

7. Interface Specifications

7.1. Jupyter Notebook Interface

The primary user interface is a Jupyter Notebook (.ipynb) organized into sequential cells corresponding to each pipeline stage. The interface supports:

- Cell-by-cell execution with visible intermediate outputs (DataFrame previews, shape logs)
- Inline visualization of confusion matrix heatmap and ROC curve
- A dedicated prediction cell accepting a Python dictionary of customer attributes
- Markdown cells documenting each stage for academic reviewers

7.2. Prediction Input Format

The prediction interface accepts a Python dictionary with the following structure:

Key	Example Value	Data Type
CreditScore	650	int
Geography	'France'	str
Gender	'Female'	str
Age	42	int
Tenure	3	int
Balance	75000.0	float
NumOfProducts	2	int
HasCrCard	1	int (0 or 1)
IsActiveMember	1	int (0 or 1)
EstimatedSalary	60000.0	float

7.3. Prediction Output Format

Output Field	Type	Description
churn_label	int	0 = Not likely to churn; 1 = Likely to churn
churn_probability	float	Probability score for churn class (0.0 – 1.0)
recommendation	str	Human-readable retention action message

8. State and Session Management

The system operates in a stateless, session-independent manner. Each notebook execution creates fresh in-memory objects for all modules. No session cookies, tokens, or server-side state is maintained between executions.

Persistence is achieved exclusively through file I/O: the trained model is serialized to disk using joblib, and the scaler object is saved alongside it. This allows the PredictionEngine to reload a trained state without re-running the full training pipeline.

9. Caching Strategy

No runtime caching is implemented in the current version. All data transformations are performed during each execution. The following caching improvements are recommended for future iterations:

- Persist the preprocessed DataFrame to a local Parquet file to skip reprocessing on subsequent runs
- Cache the fitted StandardScaler as a separate .pkl file alongside the model
- Use Python's functools.lru_cache decorator on pure utility functions (e.g., ratio computations) if batch prediction volume increases significantly

10. Non-Functional Requirements – Implementation Details

10.1. Security

- All data files are accessed via validated absolute paths; no user-supplied path traversal is permitted
- No sensitive customer data is transmitted externally; all processing is local
- Model serialization uses joblib; untrusted model files should not be loaded in production without integrity verification

10.2. Performance

- Training on 10,000 records completes in under 5 seconds on standard hardware (tested on Intel Core i5, 8GB RAM)
- Single-record prediction latency is under 50 milliseconds including preprocessing
- Batch prediction of 1,000 records completes in under 2 seconds

10.3. Scalability

- The pipeline architecture supports replacement of pandas with PySpark for datasets exceeding 1 million rows
- The ChurnModel class interface is model-agnostic; any sklearn-compatible estimator can be swapped in

10.4. Reliability

- All preprocessing transformations are deterministic given the same input data
- Random state is fixed at 42 in all stochastic operations to guarantee reproducible results

- Model performance is validated using stratified train-test split to prevent data leakage

10.5. Usability

- All class methods include Python docstrings with parameter types and return descriptions
- Each notebook cell produces at least one visible output (print statement or plot) for traceability
- Error messages include the module name, method name, and descriptive remediation hint

11. References

- Scikit-learn Documentation – <https://scikit-learn.org/stable/>
- Pandas Documentation – <https://pandas.pydata.org/docs/>
- NumPy Documentation – <https://numpy.org/doc/>
- Joblib Documentation – <https://joblib.readthedocs.io/>
- Kaggle Dataset – Churn Modelling Dataset (Bank Customer Churn Prediction)
- Python Logging Module – <https://docs.python.org/3/library/logging.html>
- Machine Learning Best Practices – Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow (Géron, 2019)